

Timestep sensitivities: CTSE and OTI real matrices

Contents

1	Background	2
1.1	Automatic differentiation	2
1.2	Finite differences and their limitations	2
1.3	First derivatives with CTSE	3
1.3.1	Worked example: $f(t) = \exp(t^2)$ by complex step	4
1.4	OTI arithmetic and Taylor coefficients	6
1.4.1	Worked example: $f(t) = \exp(t^2)$ in OTI arithmetic	7
1.5	Cauchy–Riemann matrix representation	7
1.5.1	Univariate functions in the representation	8
1.5.2	Worked example: $f(t) = \exp(t^2)$ in the CR representation	9
1.5.3	Matrices of OTI numbers	10
1.5.4	Worked example: $F(t) = \exp(tH)$ for a matrix argument	11
2	Implementation	12
2.1	Original response calculation	12
2.2	CTSE: complex input and derivative extraction	12
2.3	OTI–CR: real block assembly and coefficient extraction	13
2.4	Comparison and verification code	14
3	Results	15
3.1	Function values and derivative agreement	15
3.2	Derivative profiles and step-size dependence	16
A	Multivariate OTI numbers and their real matrix representation	17
A.1	Algebra, seeding, and mixed derivatives	17
A.2	Building the matrix algebra from its generators	18
A.3	Worked example with pure and mixed derivatives	20
A.4	General matrices with multivariate OTI entries	21

1 Background

This section introduces differentiation methods for a generic scalar or vector-valued function $F(t)$ of one real parameter. Section 2 applies these methods to the example problem in `MatrixExponentialTest.jl`, identifying the changes to the original numerical calculation. Section 3 presents the comparisons.

1.1 Automatic differentiation

Automatic differentiation (AD), also known as *algorithmic differentiation* or *autodiff*, is a technique for computing numerical derivatives of functions expressed as computer programs with high accuracy. AD is used in machine learning, optimization, and scientific computing, where efficient and accurate gradients are required [1, 2]. It is particularly useful when a calculation is too complex for practical manual differentiation or when an explicit derivative formula is unavailable.

AD decomposes a function evaluation into arithmetic operations and elementary functions with known derivatives, forming an *evaluation trace*. This trace can be represented as a computational graph: nodes describe operations and intermediate values, while directed edges describe their dependencies. Applying the chain rule along these dependencies produces derivatives of the complete calculation [3]. For example, the evaluation of $f(t) = \exp(t^2)$, used again in Sections 1.3.1, 1.4.1, and 1.5.2, can be written as $v_1 = t^2$ followed by $v_2 = \exp(v_1)$. Propagating derivatives gives

$$\frac{dv_1}{dt} = 2t, \quad \frac{dv_2}{dt} = \exp(v_1) \frac{dv_1}{dt} = 2t \exp(t^2).$$

The derivative is evaluated numerically without constructing a global symbolic expression or approximating a limit with a finite perturbation. For a differentiable evaluation trace, AD introduces no finite-difference truncation error, although floating-point rounding and conditioning still affect accuracy. Forward mode propagates derivatives from inputs to outputs; reverse mode propagates sensitivities from outputs back to inputs [3].

1.2 Finite differences and their limitations

Where AD propagates derivatives through the evaluation trace, finite differences (FD) approximate them from function values at nearby real inputs. For a sufficiently smooth function F and a real step h ,

$$F'(t) \approx \frac{F(t+h) - F(t)}{h}, \quad F'(t) \approx \frac{F(t+h) - F(t-h)}{2h}. \quad (1)$$

The forward and centered formulas in Equation (1) have truncation errors of order h and h^2 , respectively. Reducing h reduces these errors, but eventually the subtraction of nearly equal function values amplifies roundoff relative to the small difference. Noise or solver errors in function values are also amplified by division by h . Consequently, a smaller step does not always give a more accurate derivative [5]. Higher-order FD formulas amplify evaluation errors through division by higher powers of h , and suitable steps depend on the derivative order and the function's scales. FD requires only function evaluations, making it a useful independent check, but its accuracy depends on the stencil and step-size choice.

Truncation and roundoff in a worked example. Figure 1 illustrates this tradeoff for $f(t) = \exp(t^2)$ at $t = 1$, the same example used in Sections 1.3.1, 1.4.1, and 1.5.2. For the centered formula in Equation (1),

$$D_c(h) = \frac{\exp((1+h)^2) - \exp((1-h)^2)}{2h} = 2e + \frac{10}{3}eh^2 + O(h^4). \quad (2)$$

The truncation error $(10/3)eh^2$ falls as h is reduced, while cancellation in the numerator grows roughly as ϵ_{mach}/h , where ϵ_{mach} is the spacing from 1 to the next larger Float64 number. The competing trends give a U-shaped error curve.

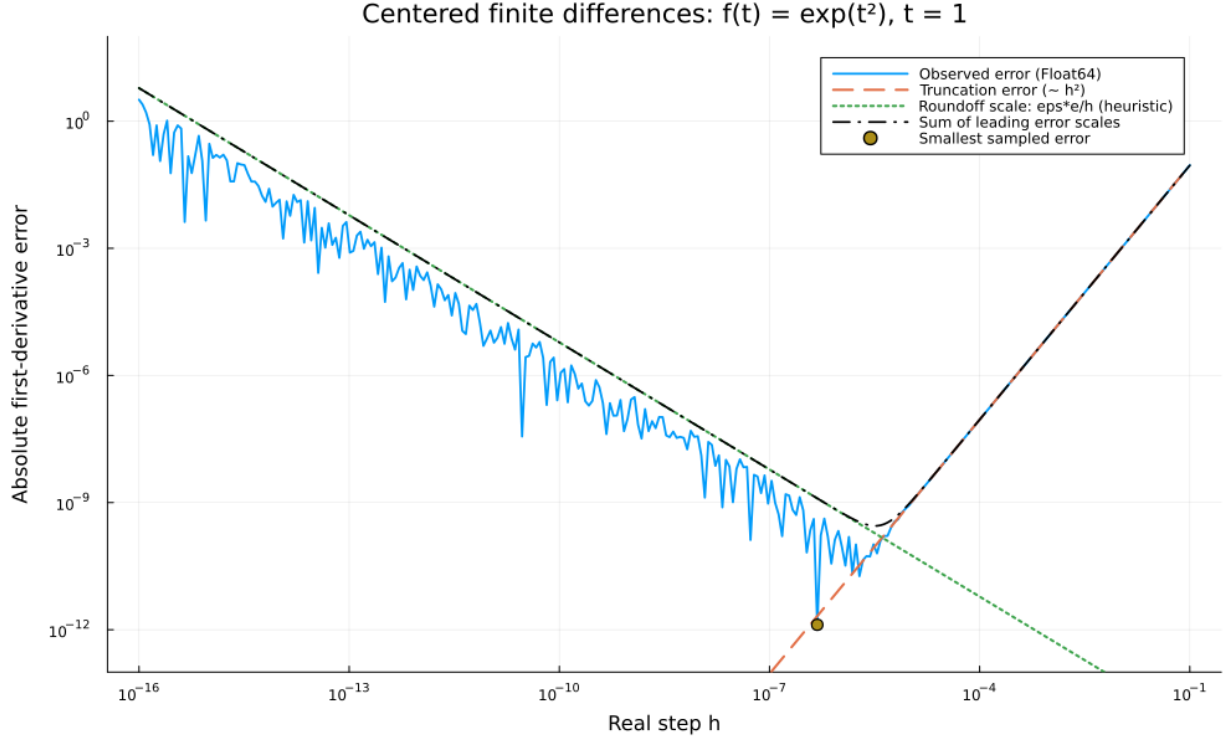


Figure 1: Centered finite-difference error for $f(t) = \exp(t^2)$ at $t = 1$. The observed error compares the Float64 derivative with a 256-bit reference $2e$. Truncation error is evaluated with the same centered stencil in 256-bit arithmetic. The roundoff scale and summed error model illustrate the competing trends; they are not bounds on the observed error.

The smallest error lies between the two regimes, but its location depends on the function, the evaluation point, and the arithmetic, so no rule fixes it in advance. Choosing a step is therefore a tuning exercise, and that difficulty is the practical motivation for the alternatives that follow: CTSE removes the cancellation, and OTI arithmetic removes the step size altogether. Figure 2 contrasts the CTSE behaviour for the same function and evaluation point. The figure and raw error data can be regenerated from the repository root:

```
julia --project=. docs/fd_worked_example.jl
```

1.3 First derivatives with CTSE

Complex Taylor series expansion (CTSE), also called the complex-step method, evaluates the response at a complex input $t + ih$, where $i^2 = -1$ and $h > 0$ is a small real step [6]. If F is analytic and real-valued on the real axis,

$$F(t + ih) = F(t) + ihF'(t) - \frac{h^2}{2}F''(t) - \frac{ih^3}{6}F'''(t) + O(h^4).$$

Taking the imaginary part and dividing by h gives

$$\boxed{F'(t) \approx \frac{\text{Im } F(t + ih)}{h}}, \quad \frac{\text{Im } F(t + ih)}{h} = F'(t) - \frac{h^2}{6}F'''(t) + O(h^4). \quad (3)$$

The real part supplies $F(t)$ with an $O(h^2)$ error. Unlike a finite difference, Equation (3) does not subtract nearly equal response values, allowing a very small h . CTSE is a numerical derivative approximation; OTI arithmetic, introduced in Section 1.4, propagates Taylor coefficients algebraically.

The evaluated function must admit an analytic extension near the real input. Operations such as absolute values, conjugation, or premature removal of imaginary components can invalidate the method. Standard CTSE provides the first derivative in Equation (3); one evaluation does not separate the higher derivatives also present in its Taylor expansion. Other complex-step extensions exist, including multicomplex numbers for high-order derivatives and hyper-dual numbers for exact second derivatives [7, 8], but are outside the scope of this comparison.

1.3.1 Worked example: $f(t) = \exp(t^2)$ by complex step

Consider the composition $v = t^2$, $f = \exp(v)$ at $t_0 = 1$. Sections 1.4.1 and 1.5.2 evaluate this same function using OTI arithmetic and its CR representation. Direct differentiation supplies reference values:

$$f(1) = e, \quad f'(1) = 2e, \quad f''(1) = 6e. \quad (4)$$

Here $e = \exp(1)$ is Euler's number. For CTSE, first seed the input as $\tilde{t} = 1 + ih$. Then evaluate the original two operations with complex arithmetic, retaining the imaginary component at each step:

$$\begin{aligned} \tilde{v} &= (1 + ih)^2 = 1 - h^2 + 2ih, \\ \tilde{f} &= \exp(\tilde{v}) = \exp(1 - h^2) [\cos(2h) + i \sin(2h)]. \end{aligned}$$

The real and imaginary parts therefore give

$$f(1) \approx \exp(1 - h^2) \cos(2h), \quad f'(1) \approx \exp(1 - h^2) \frac{\sin(2h)}{h}. \quad (5)$$

Expanding the extracted derivative in Equation (5) gives

$$\exp(1 - h^2) \frac{\sin(2h)}{h} = 2e - \frac{10}{3}eh^2 + O(h^4). \quad (6)$$

The leading term matches Equation (2) in magnitude and differs only in sign. This is not particular to the example: for any analytic F , the centered difference carries $+h^2 F'''(t)/6$ and the complex step $-h^2 F'''(t)/6$. The two methods therefore share the same truncation error, and what separates them is the other error source. The centered difference forms $F(t + h) - F(t - h)$, a subtraction of increasingly close values, so reducing h eventually destroys accuracy through cancellation. The complex step takes an imaginary part rather than a difference, so no such subtraction occurs and h may be reduced far below the finite-difference optimum.

Equation (6) predicts second-order convergence as h decreases, until numerical rounding dominates. Table 1 presents the computed function values and first derivatives for selected steps. Figure 2 shows the corresponding convergence and the leading truncation-error magnitude $(10/3)eh^2$. The original operations are evaluated once per complex input; neither a symbolic derivative nor a difference of nearby function values is needed.

Numerical results for the worked example. The evaluations use Julia’s ordinary `Float64` complex arithmetic. The reference values are $f(1) = e \approx 2.718281828459$ and $f'(1) = 2e \approx 5.436563656918$. To distinguish rounding error from truncation error, absolute errors are computed against a 256-bit evaluation of $2e$, using the full stored derivative rather than its printed digits.

Table 1: CTSE evaluations of $f(t) = \exp(t^2)$ at $t = 1$. Function values and derivatives are printed to twelve decimal places; errors use the full stored values and a high-precision reference.

Step h	$\operatorname{Re} \tilde{f}$	$\operatorname{Im} \tilde{f}/h$	Absolute derivative error
1.000×10^{-1}	2.637588959484	5.346657516342	8.991×10^{-2}
1.000×10^{-2}	2.717466429984	5.435657633647	9.060×10^{-4}
1.000×10^{-3}	2.718273673622	5.436554595986	9.061×10^{-6}
1.000×10^{-4}	2.718281746911	5.436563566309	9.061×10^{-8}
1.000×10^{-6}	2.718281828451	5.436563656909	9.061×10^{-12}
1.000×10^{-8}	2.718281828459	5.436563656918	2.891×10^{-16}
1.000×10^{-12}	2.718281828459	5.436563656918	2.891×10^{-16}
1.000×10^{-20}	2.718281828459	5.436563656918	2.891×10^{-16}

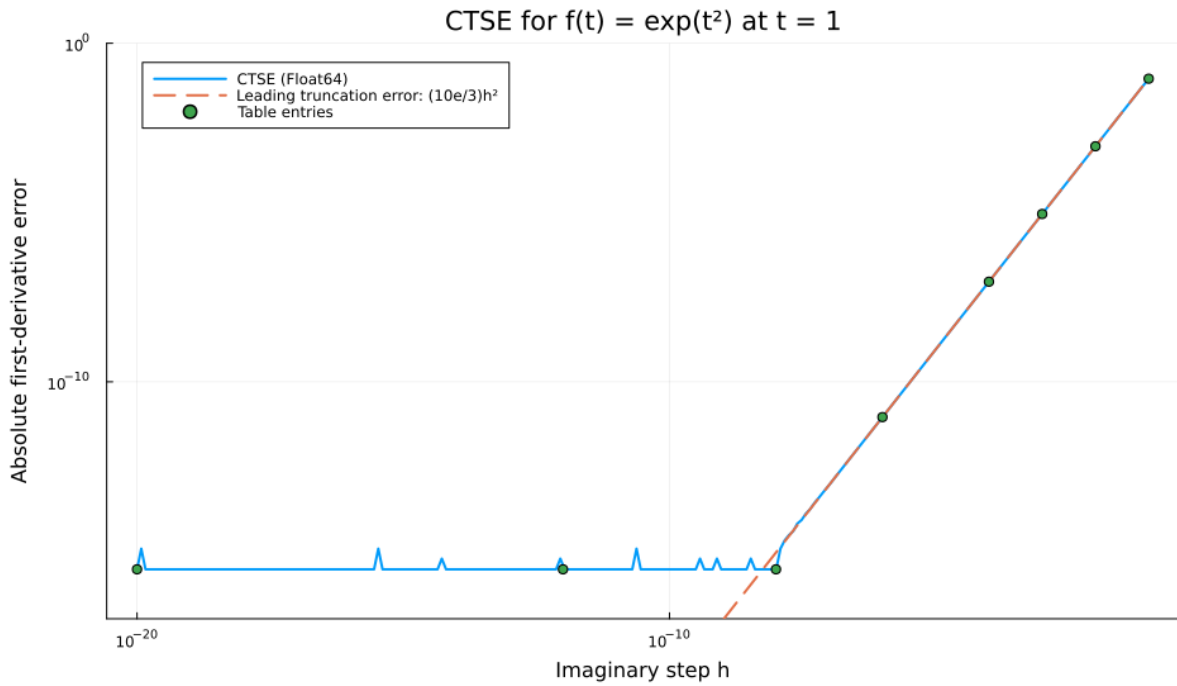


Figure 2: CTSE first-derivative convergence for the worked example. The dashed line is the leading truncation error from Equation (6); markers identify the steps in Table 1.

Figure 2 shows the error initially decreasing in proportion to h^2 . For sufficiently small steps, the error reaches a floating-point rounding level and no longer decreases systematically. Unlike real finite differences, reducing h further does not introduce subtractive cancellation in the derivative extraction for this example. The table, figure, and raw CSV data can be regenerated from the repository root:

```
julia --project=. docs/ctse_worked_example.jl
```

1.4 OTI arithmetic and Taylor coefficients

For one variable and truncation order p , consider two order-truncated imaginary (OTI) numbers [10]:

$$a = \sum_{j=0}^p a_j \varepsilon^j, \quad b = \sum_{j=0}^p b_j \varepsilon^j, \quad a_j, b_j \in \mathbb{R}, \quad \varepsilon^{p+1} = 0. \quad (7)$$

The symbol ε is *nilpotent*: sufficiently high powers vanish. At $p = 1$, Equation (7) reduces to the *dual numbers* $a_0 + a_1 \varepsilon$ with $\varepsilon^2 = 0$, which carry a value and one derivative through a forward-mode evaluation [8]. OTI numbers extend that same construction to order p , carrying p derivatives at once.

The defining algebraic relation also distinguishes OTI arithmetic from the CTSE method in Section 1.3. Under the complex step, $i^2 = -1$ folds the third-order Taylor term back into the imaginary part, which is the origin of the $O(h^2)$ truncation error in Equation (3). In OTI arithmetic, powers above order p vanish, while retained powers remain distinct rather than contributing to a lower-order coefficient. The retained coefficients are therefore exact in the algebra rather than approximations improved by shrinking a step. The name retains *imaginary* because the construction parallels $\mathbb{C}$ by adjoining a new unit to the reals, but imposes the nilpotency relation $\varepsilon^{p+1} = 0$, and because extracting a coefficient plays the role of taking an imaginary part, as the notation $\text{Im}_{\varepsilon^k}$ in Equation (11) reflects.

Addition combines coefficients of the same power:

$$a + b = \sum_{j=0}^p (a_j + b_j) \varepsilon^j. \quad (8)$$

At order two, Equation (8) becomes

$$a + b = (a_0 + b_0) + (a_1 + b_1) \varepsilon + (a_2 + b_2) \varepsilon^2.$$

Multiplication collects terms of equal degree and discards terms above order p :

$$ab = \sum_{k=0}^p \left(\sum_{j=0}^k a_j b_{k-j} \right) \varepsilon^k.$$

For example, at order two, $a = a_0 + a_1 \varepsilon + a_2 \varepsilon^2$ and $b = b_0 + b_1 \varepsilon + b_2 \varepsilon^2$ give

$$ab = a_0 b_0 + (a_0 b_1 + a_1 b_0) \varepsilon + (a_0 b_2 + a_1 b_1 + a_2 b_0) \varepsilon^2.$$

An OTI scalar type can support *operator overloading* [4]: operations such as addition and multiplication are defined to combine coefficients according to these rules. Elementary functions such as $\exp$ and $\sin$ can likewise be overloaded using their Taylor expansions. For $a = a_0 + \sum_{r=1}^p a_r \varepsilon^r$, the expansion about its real part a_0 is evaluated at the full OTI input as

$$g\left(a_0 + \sum_{r=1}^p a_r \varepsilon^r\right) = \sum_{j=0}^p \frac{g^{(j)}(a_0)}{j!} \left(\sum_{r=1}^p a_r \varepsilon^r\right)^j. \quad (9)$$

Powers above p vanish because the displacement has zero constant coefficient and $\varepsilon^{p+1} = 0$. A calculation composed of supported operations then propagates coefficients through the same sequence of function evaluations. Section 1.5 provides an equivalent real matrix representation; the implementation in Section 2.3 uses that representation rather than defining an overloaded OTI scalar type.

Seeding the input as $t + \varepsilon$ gives, for an analytic function,

$$F(t + \varepsilon) = F(t) + F'(t) \varepsilon + \frac{F''(t)}{2!} \varepsilon^2 + \cdots + \frac{F^{(p)}(t)}{p!} \varepsilon^p. \quad (10)$$

Define $\text{Im}_{\varepsilon^k}(a) := a_k$ to mean extraction of the coefficient of ε^k . Equation (10) therefore yields the unscaled derivative

$$\boxed{F^{(k)}(t) = k! \text{Im}_{\varepsilon^k}(F(t + \varepsilon))}, \quad 1 \leq k \leq p. \quad (11)$$

The constant coefficient is $F(t)$.

1.4.1 Worked example: $f(t) = \exp(t^2)$ in OTI arithmetic

Use the function and evaluation point of Section 1.3.1, now with truncation order $p = 2$. Seed $t^* = 1 + \varepsilon$ and impose $\varepsilon^3 = 0$. The square is obtained by multiplying two OTI numbers $a = b = 1 + \varepsilon$:

$$v^* = t^* t^* = 1 + 2\varepsilon + \varepsilon^2.$$

The coefficients of v^* are therefore 1, 2, and 1. The next operation is $g(v) = \exp(v)$. Its input is the OTI number v^* , which is decomposed into a real part and a perturbation expressed in powers of ε :

$$v^* = v_0 + 2\varepsilon + \varepsilon^2, \quad v_0 = 1. \quad (12)$$

The Taylor series expansion (TSE) is taken *about the real part* v_0 . Before substituting the OTI input, write the expansion at a general argument s :

$$g(s) = g(v_0) + g'(v_0)(s - v_0) + \frac{g''(v_0)}{2!}(s - v_0)^2 + \sum_{j=3}^{\infty} \frac{g^{(j)}(v_0)}{j!}(s - v_0)^j. \quad (13)$$

Now evaluate Equation (13) at the full OTI argument $s = v^* = v_0 + 2\varepsilon + \varepsilon^2$. The displacement from the expansion point is $(v_0 + 2\varepsilon + \varepsilon^2) - v_0 = 2\varepsilon + \varepsilon^2$, giving

$$\begin{aligned} g(v_0 + 2\varepsilon + \varepsilon^2) &= g(v_0) + g'(v_0)[(v_0 + 2\varepsilon + \varepsilon^2) - v_0] \\ &\quad + \frac{g''(v_0)}{2!}[(v_0 + 2\varepsilon + \varepsilon^2) - v_0]^2 \\ &= g(v_0) + g'(v_0)(2\varepsilon + \varepsilon^2) + \frac{g''(v_0)}{2!}(2\varepsilon + \varepsilon^2)^2. \end{aligned} \quad (14)$$

Terms of degree three and higher vanish because $(2\varepsilon + \varepsilon^2)^3 = 0$ in the order-two algebra. Equation (14) is therefore exact in that algebra, rather than an approximation obtained by assuming a small real perturbation. The function and its elementary derivatives are evaluated at the real scalar v_0 ; the powers of the displacement use OTI arithmetic.

For the exponential, $g(1) = g'(1) = g''(1) = e$. Substituting these real values and the perturbation from Equation (12) gives

$$\exp(1 + 2\varepsilon + \varepsilon^2) = e + e(2\varepsilon + \varepsilon^2) + \frac{e}{2!}(2\varepsilon + \varepsilon^2)^2.$$

The remaining multiplication is truncated according to $\varepsilon^3 = 0$:

$$(2\varepsilon + \varepsilon^2)^2 = 4\varepsilon^2 + 4\varepsilon^3 + \varepsilon^4 = 4\varepsilon^2.$$

Collecting equal powers of ε yields the output of the overloaded exponential:

$$f^* = e + 2e\varepsilon + e\varepsilon^2 + 2e\varepsilon^2 = e + 2e\varepsilon + 3e\varepsilon^2. \quad (15)$$

The exponential's overload uses its known derivatives at the real part; it does not require a manually derived formula for the derivative of the full composition $f(t) = \exp(t^2)$. Its coefficients are e , $2e$, and $3e$. Applying Equation (11) gives

$$f(1) = e, \quad f'(1) = 1!(2e) = 2e, \quad f''(1) = 2!(3e) = 6e.$$

The second-order coefficient is $3e$, whereas the second derivative is $6e$. These results match Equation (4) exactly in the truncated algebra, without choosing a numerical step size. The evaluation trace still consists of a multiplication followed by an exponential; the definitions of those operations propagate the coefficients. Section 1.5.2 performs this same calculation using real matrices.

1.5 Cauchy–Riemann matrix representation

The univariate OTI algebra of Section 1.4 has one generator, ε , and truncation order p . Its basis is $(1, \varepsilon, \dots, \varepsilon^p)$, with $\varepsilon^{p+1} = 0$. To represent this algebra using real matrices, we first construct a matrix N representing multiplication by ε . Higher powers of ε are then represented by powers of N , and a general OTI number is represented by the same linear combination of these matrices. This is the generalized Cauchy–Riemann representation used here [10].

Construct the generator matrix from its action on the basis. At order two, multiplying each monomial in the ordered basis by ε gives

$$\varepsilon(1, \varepsilon, \varepsilon^2) = (\varepsilon, \varepsilon^2, 0). \quad (16)$$

For each output, write its coordinates in the order $(1, \varepsilon, \varepsilon^2)$ and place that vector in the corresponding column of N . The first output, ε , gives the first column $(0, 1, 0)^T$; the second output, ε^2 , gives $(0, 0, 1)^T$; the last output is zero. Thus

$$N = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad N^2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad N^3 = 0.$$

The product N^2 represents multiplication by ε^2 . At general order p , the same construction gives a $(p+1) \times (p+1)$ matrix with ones immediately below the diagonal and zeros elsewhere. Multiplication shifts each monomial to the next power, until the result vanishes at degree $p+1$; consequently $N^{p+1} = 0$.

Build the representation of a general number. For $a = \sum_{j=0}^p a_j \varepsilon^j$, replace 1 by I_{p+1} and each ε^j by N^j , keeping the real coefficients:

$$\boxed{a = \sum_{j=0}^p a_j \varepsilon^j \mapsto C(a) = \sum_{j=0}^p a_j N^j} \quad (17)$$

$$= a_0 I_{p+1} + a_1 N + \cdots + a_p N^p$$

Here $N^0 = I_{p+1}$. In particular, at order two,

$$a = a_0 + a_1 \varepsilon + a_2 \varepsilon^2 \mapsto C(a) = a_0 I_3 + a_1 N + a_2 N^2$$

$$= \begin{bmatrix} a_0 & 0 & 0 \\ a_1 & a_0 & 0 \\ a_2 & a_1 & a_0 \end{bmatrix}. \quad (18)$$

There is one generator matrix N , while $I_{p+1}, N, \dots, N^p$ form a basis for the represented algebra. Appendix A.2 extends this construction to several independent generators.

Carry out OTI arithmetic using matrix operations. To use this representation in a calculation, addition and multiplication must reproduce the OTI rules of Section 1.4. Addition combines the coefficients of equal powers of N . Multiplication uses $N^i N^j = N^{i+j}$, with powers above p equal to zero, exactly as for ε . Consequently, for any two OTI numbers a and b , $C(a+b) = C(a) + C(b)$ and $C(ab) = C(a)C(b)$. The resulting correspondence is

$$\begin{aligned} 1 &\mapsto I_{p+1}, & a &\mapsto C(a), \\ \varepsilon &\mapsto N, & a+b &\mapsto C(a) + C(b), \\ \varepsilon^{p+1} = 0 &\mapsto N^{p+1} = 0, & ab &\mapsto C(a)C(b). \end{aligned} \quad (19)$$

The seed $t + \varepsilon$ of Section 1.4 therefore becomes $C(t + \varepsilon) = tI_{p+1} + N$. To recover the OTI result after a matrix calculation, read its coefficients from the first column, as illustrated in Equation (18): the coefficient of ε^k occupies entry $k+1$. For a seeded function evaluation, these coefficients give derivatives through the factorial scaling in Equation (11). The convention adopted here uses lower triangular matrices and coefficients ordered by increasing degree. An upper triangular convention would require corresponding changes in extraction.

1.5.1 Univariate functions in the representation

Because C preserves sums and products, it preserves every polynomial and therefore every convergent power series. For an analytic f ,

$$f(C(a)) = C(f(a)), \quad (20)$$

where the left-hand side is the *matrix* function of $C(a)$ [9]. Evaluating a matrix function of the representation therefore returns the representation of the OTI result, and no OTI scalar type is required.

At first order, seed the input as $t + \varepsilon$ with $\varepsilon^2 = 0$. The real number t contributes tI_2 and the seed contributes N , so

$$C(t + \varepsilon) = tI_2 + N = \begin{bmatrix} t & 0 \\ 1 & t \end{bmatrix}.$$

Take $f = \exp$. By Equation (10) at $p = 1$, the OTI result is $\exp(t + \varepsilon) = e^t + e^t \varepsilon$, so Equation (20) returns its representation directly:

$$\exp(tI_2 + N) = e^t I_2 + e^t N = \begin{bmatrix} e^t & 0 \\ e^t & e^t \end{bmatrix}.$$

Its first column is $[e^t, e^t]^T$, which is $[f(t), f'(t)]$. Because $\exp$ is its own derivative, this example on its own cannot separate the method from a coincidence.

Take $f = \sin$ instead. Now $\sin(t + \varepsilon) = \sin t + \cos(t) \varepsilon$, and the same substitution gives

$$\sin(tI_2 + N) = \sin(t) I_2 + \cos(t) N = \begin{bmatrix} \sin t & 0 \\ \cos t & \sin t \end{bmatrix}.$$

Its first column is $[\sin t, \cos t]^T$, again $[f(t), f'(t)]$, now with the value and the derivative distinct.

Nothing about the seed changes with the truncation order. It is the same OTI number $t + \varepsilon$ throughout; raising p only enlarges the matrices that carry it, since $\varepsilon^{p+1} = 0$ requires $N^{p+1} = 0$ and N is $(p+1) \times (p+1)$. At order p the seed is therefore represented by

$$C(t + \varepsilon) = tI_{p+1} + N = \begin{bmatrix} t & 0 & \cdots & 0 \\ 1 & t & \ddots & \vdots \\ 0 & \ddots & \ddots & 0 \\ 0 & \cdots & 1 & t \end{bmatrix} \in \mathbb{R}^{(p+1) \times (p+1)}, \quad (21)$$

with t on the diagonal, ones on the first subdiagonal, and zeros elsewhere; the 2×2 matrix above is the case $p = 1$. Applying Equation (20) to Equation (21),

$$C(f(t + \varepsilon)) \text{ has first column } \left[f(t), \frac{f'(t)}{1!}, \frac{f''(t)}{2!}, \dots, \frac{f^{(p)}(t)}{p!} \right]^T. \quad (22)$$

These entries are Taylor coefficients, not derivatives. At first order the distinction is invisible because $1! = 1$; at second order it is not. For $f = \exp$ at order two,

$$\exp \begin{bmatrix} t & 0 & 0 \\ 1 & t & 0 \\ 0 & 1 & t \end{bmatrix} = e^t \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ \frac{1}{2} & 1 & 1 \end{bmatrix},$$

whose first column is $[e^t, e^t, e^t/2]^T$. The second derivative follows from the factorial scaling of Equation (11), as $2! (e^t/2) = e^t$.

The Taylor expansion about tI_{p+1} terminates because $N^{p+1} = 0$. The truncation order is fixed by the size of the matrix representing the seed.

1.5.2 Worked example: $f(t) = \exp(t^2)$ in the CR representation

Represent the order-two seed $t^* = 1 + \varepsilon$ from Section 1.4.1 by the ordinary real matrix

$$T = C(t^*) = I_3 + N = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

The same evaluation trace becomes a matrix multiplication followed by a matrix exponential. The first operation gives

$$V = T^2 = I_3 + 2N + N^2 = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Thus $V = C(1 + 2\varepsilon + \varepsilon^2)$: its first column contains exactly the intermediate coefficients from Section 1.4.1. For the second operation, write $V = I_3 + K$ with $K = 2N + N^2$. Since $K^2 = 4N^2$ and $K^3 = 0$, the matrix exponential reduces to

$$W = \exp(V) = e \left(I_3 + K + \frac{K^2}{2} \right) = e \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 2 & 1 \end{bmatrix}. \quad (23)$$

The first column of W , equivalently $W[1, 0, 0]^T$, is $[e, 2e, 3e]^T$. It contains the Taylor coefficients, so multiplying the third entry by $2!$ yields the second derivative:

$$f(1) \approx 2.718281828459, \quad f'(1) \approx 5.436563656918, \quad f''(1) \approx 16.309690970754.$$

These are the same results as Equation (15). No OTI scalar type is required: an ordinary real matrix multiplication computes T^2 , and a real matrix-exponential routine computes $\exp(V)$. These are matrix operations, not entrywise squaring or exponentiation. The matrix representation changes how the coefficients are stored and propagated; the factorial extraction rule remains the same as in Section 1.4.

1.5.3 Matrices of OTI numbers

Sections 1.5.1 and 1.5.2 applied functions to a single OTI number. The calculation in Section 2 needs the exponential of a matrix, whose argument becomes a matrix of OTI numbers once the duration is seeded.

The same generator matrix N also gives this representation. First collect the coefficients at each order into real matrices:

$$M = M_0 + M_1\varepsilon + \cdots + M_p\varepsilon^p, \quad M_j \in \mathbb{R}^{m \times m}.$$

The scalar construction in Equation (17) now becomes a block construction: each scalar coefficient a_j is replaced by its coefficient matrix M_j . At order two, this gives

$$\mathcal{C}(M) = \begin{bmatrix} M_0 & 0 & 0 \\ M_1 & M_0 & 0 \\ M_2 & M_1 & M_0 \end{bmatrix} \in \mathbb{R}^{3m \times 3m}, \quad (24)$$

where M_j holds the order- j coefficient of each entry of the $m \times m$ matrix M , and each 0 is the $m \times m$ zero block.

For example, take $m = 2$, $p = 1$, and $M = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ with $a = a_0 + a_1\varepsilon$ and b, c, d likewise. Then $M_0 = \begin{bmatrix} a_0 & b_0 \\ c_0 & d_0 \end{bmatrix}$, $M_1 = \begin{bmatrix} a_1 & b_1 \\ c_1 & d_1 \end{bmatrix}$, and

$$\mathcal{C}(M) = \begin{bmatrix} M_0 & 0 \\ M_1 & M_0 \end{bmatrix} = \left[\begin{array}{cc|cc} a_0 & b_0 & 0 & 0 \\ c_0 & d_0 & 0 & 0 \\ \hline a_1 & b_1 & a_0 & b_0 \\ c_1 & d_1 & c_0 & d_0 \end{array} \right].$$

At truncation order p the assembled matrix is $(p+1)m \times (p+1)m$, and at $m = 1$ it is $\mathcal{C}(a)$ again.

The same block construction can be written using the *Kronecker product*, denoted by $\otimes$. This product replaces each scalar entry of its first matrix by that scalar times the entire second matrix. For example,

$$\begin{bmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \end{bmatrix} \otimes B = \begin{bmatrix} u_{11}B & u_{12}B \\ u_{21}B & u_{22}B \end{bmatrix}.$$

Using the generator matrix N constructed in Section 1.5, the representation at arbitrary order p is

$$\boxed{\mathcal{C}(M) = \sum_{j=0}^p N^j \otimes M_j}, \quad N^0 = I_{p+1}. \quad (25)$$

Here N acts on the order of the coefficients, while each M_j acts on the original matrix dimension. The identity places M_0 in every diagonal block; N places M_1 on the first block subdiagonal; N^2 places M_2 on the second block subdiagonal, and so on. Thus, at order two,

$$\mathcal{C}(M) = I_3 \otimes M_0 + N \otimes M_1 + N^2 \otimes M_2 = \begin{bmatrix} M_0 & 0 & 0 \\ M_1 & M_0 & 0 \\ M_2 & M_1 & M_0 \end{bmatrix},$$

which is exactly Equation (24). At first order this reduces to $\mathcal{C}(M) = I_2 \otimes M_0 + N \otimes M_1$. Section 1.5.4 applies this expression with $M_0 = tH$ and $M_1 = H$. Appendix A extends the construction to multiple parameters and mixed derivatives.

The dictionary of Equation (19) carries over, with products preserved in order since matrices need not commute, and with it any function given by a convergent series, the matrix exponential included. A real exponential routine applied to Equation (24) therefore returns the representation of the OTI result.

1.5.4 Worked example: $F(t) = \exp(tH)$ for a matrix argument

Take the constant real matrix and matrix-valued function

$$H = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad F(t) = \exp(tH) = \begin{bmatrix} \cos t & \sin t \\ -\sin t & \cos t \end{bmatrix},$$

and use the first-order real seed representation from Section 1.5.1:

$$T = tI_2 + N = \begin{bmatrix} t & 0 \\ 1 & t \end{bmatrix}, \quad N = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad N^2 = 0. \quad (26)$$

The seed matrix T carries the value and first-order information, while H acts on the two components of the original matrix problem. Applying the Kronecker representation in Equation (25), with $M_0 = tH$ and $M_1 = H$, gives

$$\mathcal{M} = T \otimes H = (tI_2 + N) \otimes H = I_2 \otimes (tH) + N \otimes H. \quad (27)$$

The two terms in Equation (27) produce the blocks explicitly:

$$I_2 \otimes (tH) = \begin{bmatrix} tH & 0_{2 \times 2} \\ 0_{2 \times 2} & tH \end{bmatrix}, \quad N \otimes H = \begin{bmatrix} 0_{2 \times 2} & 0_{2 \times 2} \\ H & 0_{2 \times 2} \end{bmatrix}.$$

Adding them places tH in both diagonal blocks and H in the lower-left block, recovering the pattern of Section 1.5.3:

$$\mathcal{M} = \begin{bmatrix} tH & 0_{2 \times 2} \\ H & tH \end{bmatrix} = \left[\begin{array}{cc|cc} 0 & t & 0 & 0 \\ -t & 0 & 0 & 0 \\ \hline 0 & 1 & 0 & t \\ -1 & 0 & -t & 0 \end{array} \right].$$

Thus the same nilpotent matrix N used in Section 1.5.1 now acts at the block level as $N \otimes H$. Its square is zero because $(N \otimes H)^2 = N^2 \otimes H^2 = 0$. Exponentiating the ordinary real 4×4 matrix gives

$$\exp(\mathcal{M}) = \begin{bmatrix} \exp(tH) & 0 \\ H \exp(tH) & \exp(tH) \end{bmatrix},$$

whose first block column holds

$$\begin{bmatrix} \cos t & \sin t \\ -\sin t & \cos t \end{bmatrix} \quad \text{above} \quad \begin{bmatrix} -\sin t & \cos t \\ -\cos t & -\sin t \end{bmatrix}:$$

that is $F(t)$ above $F'(t)$, as differentiating F entrywise confirms. A single real matrix exponential has produced both. Section 2.3 is this calculation at full size, with H the augmented thermal matrix and the seed carried to order four.

2 Implementation

2.1 Original response calculation

The example problem in `MatrixExponentialTest.jl` evaluates a vector response $F(\Delta t)$ containing ten final voxel temperatures. The differentiation parameter is the duration Δt , nominally 50 s; the arrays A, B, e, x_k, u remain fixed. The script is retained without modification. Separate entry points, `CTSEExample.jl` and `CRExample.jl`, call reusable functions implementing the methods of Section 1.

The calculation already uses an augmented matrix exponential:

$$H = \begin{bmatrix} A & B & I_n \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad z_0 = \begin{bmatrix} x_k \\ u \\ e \end{bmatrix}, \quad F(t) = P_x \exp(tH) z_0, \quad P_x = [I_n \ 0 \ 0]. \quad (28)$$

Here P_x selects the first n entries. The top block row of $\exp(tH)$ supplies A_d, B_d , and the integrated forcing e_d , giving $F(t) = A_d x_k + B_d u + e_d$. No additional physical model is required to apply the differentiation methods. The shared `response` function in `src/OTISensitivity.jl` encapsulates this calculation as shown in Listing 1.

```
function response(A, B, e, x, u, dt)
    Ad, Bd, ed = discretize_linear_dynamics(A, B, e, dt)
    return Ad*x + Bd*u + ed
end
```

Listing 1: Reusable response evaluation.

The original final update adds e rather than e_d . Its example has $e = 0$, so this distinction does not change its result. The shared routine uses e_d and is tested with nonzero forcing. This correction and the extraction of reusable functions are supporting changes, independent of either differentiation method.

2.2 CTSE: complex input and derivative extraction

Applying Section 1.3 requires replacing t_0 by $t_0 + ih$ in Equation (28). Consequently,

$$\widetilde{M} = (t_0 + ih)H = t_0 H + ihH, \quad (29)$$

$$\widetilde{F} = P_x \exp(\widetilde{M}) z_0. \quad (30)$$

Only the duration is perturbed. The matrix H and vector z_0 retain their original real values. The scaled matrix becomes complex, while its size remains 21×21 . Since H is constant,

$$\widetilde{F} = F(t_0) + ih P_x H \exp(t_0 H) z_0 + O(h^2),$$

so the coefficient of ih is the requested derivative $F'(t_0)$.

The original routine forms a real matrix H and scales it in place (Listing 2). A complex timestep cannot be stored in that real array. The shared routine therefore allocates the scaled matrix out of place (Listing 3); its timestep argument accepts `dt::Number`, including complex values.

```
H .*= dt
G = exponential!(H)
```

Listing 2: Matrix scaling and exponential in the original script.

```
G = exponential!(augmented_dynamics(A, B) * dt)
```

Listing 3: Replacement in the shared discretization routine.

`ctse_response` in `src/SensitivityComparison.jl` then passes the complex timestep through the response routine and extracts the result as shown in Listing 4. The example uses $h = 10^{-20}$ s.

```
z = response(A, B, e, x, u, complex(dt, h))
return (; value=real.(z), derivative=imag.(z)/h)
```

Listing 4: CTSE evaluation and extraction; input validation is omitted from this excerpt.

Here z is the complex response $\tilde{F}$, not the augmented vector z_0 . The matrix-exponential routine, block extraction, and response multiplication follow the same calculation as the real evaluation. Complex components are preserved until the final real and imaginary extraction in Equation (3); no sensitivity equation is added. `CTSEExample.jl` saves these values and first derivatives to `results/ctse.csv`. Figure 4, upper-left panel, compares its first derivatives with the other methods; Figure 5, upper-left panel, assesses the imaginary step size.

2.3 OTI-CR: real block assembly and coefficient extraction

For the method of Sections 1.4 and 1.5, the seeded duration is $t_0 + \varepsilon$. At second order, the OTI matrix $M = (t_0 + \varepsilon)H$ has coefficient matrices $M_0 = t_0H$, $M_1 = H$, and $M_2 = 0$. Substitution into Equation (24) gives

$$\mathcal{H} = C(t_0 + \varepsilon) \otimes H = \begin{bmatrix} t_0H & 0 & 0 \\ H & t_0H & 0 \\ 0 & H & t_0H \end{bmatrix}. \quad (31)$$

The Kronecker product $\otimes$ replaces each scalar entry by that scalar times H . Every block is 21×21 ; Julia assembles a single real 63×63 matrix. At order p , its dimension is $21(p + 1)$.

Instead of passing t_0H to the exponential, `oti_response` in `src/OTISensitivity.jl` passes the block representation, as shown in Listing 5. The helper `oti_matrix(seed)` constructs $C(t_0 + \varepsilon)$ from the coefficients $[t_0, 1, 0, \dots, 0]$.

```
H = augmented_dynamics(A, B)
seed = zeros(promote_type(Float64, typeof(dt)), order+1)
seed[1] = dt
order > 0 && (seed[2] = 1)
G = exponential!(kron(oti_matrix(seed), H))
```

Listing 5: OTI seed and real block exponential.

Write $G_0(t) = \exp(tH)$. The second-order lifted exponential satisfies

$$\exp(\mathcal{H}) = \begin{bmatrix} G_0 & 0 & 0 \\ G'_0 & G_0 & 0 \\ G''_0/2! & G'_0 & G_0 \end{bmatrix}. \quad (32)$$

The first block column, multiplied by z_0 , contains the response coefficients. The top n entries of block j yield $F^{(j)}(t_0)/j!$.

Listing 6 shows the corresponding extraction and factorial scaling. Here $\mathbf{m}$ is the augmented dimension, $\mathbf{n}$ is the number of response entries, and $\mathbf{G}$ is the lifted exponential of Equation (32).

```
m = size(H, 1)
z = vcat(x, u, e)
coefficients = hcat([G[j*m+1:j*m+n, 1:m] * z for j in 0:order]...)
derivatives = copy(coefficients)
factor = one(eltype(derivatives))
for j in 1:order
    factor *= j
    derivatives[:, j+1] .*= factor
end
```

Listing 6: Extraction of OTI coefficients and conversion to derivatives.

Column one contains the nominal value. Column $j + 1$ of **coefficients** contains the order- j Taylor coefficient; the same column of **derivatives** contains the actual derivative. **CRExample.jl** saves orders zero through four to **results/cr.csv**. The scalar type remains real; the changes are the larger matrix passed to the solver and the extraction of its coefficient blocks. Dense storage grows quadratically and typical dense exponential work cubically with the enlarged dimension. This implementation is a baseline for method comparisons rather than an optimized solver.

2.4 Comparison and verification code

CompareSensitivities.jl runs the unchanged original in an isolated Julia module, runs the implementations in Section 2, and writes comparison CSV files and plots. The original supplies a function value only; it has no derivative implementation. The “FD baseline” function value is the unperturbed real response, not a derivative.

For derivative order $j = 1, \dots, 4$, finite differences (FD) use seven real response evaluations:

$$D_j(h) = h^{-j} \sum_{k=-3}^3 w_k^{(j)} F(t + kh), \quad \sum_{k=-3}^3 w_k^{(j)} k^q = \begin{cases} j!, & q = j, \\ 0, & 0 \leq q \leq 6, q \neq j. \end{cases} \quad (33)$$

The moment conditions in Equation (33) make the formula exact for polynomials through degree six. The weights are computed in exact rational arithmetic, then converted to floating point for use with response values. Symmetry gives truncation error $O(h^6)$ for orders one and two, and $O(h^4)$ for orders three and four. Small h increases cancellation and roundoff, roughly as $\epsilon_{\text{mach}}/h^j$. The selected step is $h_j = \max(|t|, 1)\epsilon_{\text{mach}}^{1/(j+a_j)}$ in the code’s seconds convention, where a_j is six or four respectively. At $t = 50$ s these steps are approximately 0.290, 0.552, 0.290, and 0.552 s. This is a heuristic balance, not a universal optimal step. Figure 5 presents a sweep of h to assess its effect.

An independent derivative reference follows by differentiating the augmented response $F(t) = P_x \exp(tH)z_0$ defined in Section 2.1:

$$F' = AF + Bu + e, \quad F^{(j)} = AF^{(j-1)} \quad (j \geq 2). \quad (34)$$

The reference value F is computed by diagonalizing the symmetric A and integrating each scalar mode. The forcing integral is $(\exp(\lambda t) - 1)/\lambda$, with limit t at $\lambda = 0$. This handles the insulated model’s zero mode without inverting its singular A . Errors are measured against Equation (34) and the modal reference value. Absolute errors have units K/s^j ; relative errors are $\|y - y_{\text{ref}}\|_\infty / \|y_{\text{ref}}\|_\infty$ over all ten voxels, avoiding division by an individual near-zero derivative. The comparison also directly checks CTSE against OTI-CR and FD against OTI-CR.

The scripts can be executed from the repository root using:

```
julia --project=. CTSEExample.jl
julia --project=. CRExample.jl
julia --project=. CompareSensitivities.jl
julia --project=. test/runtests.jl
```

The comparison script writes per-voxel values and derivatives, error summaries, and step sweeps to CSV files in **results/**. The figures and table in Section 3 are generated from these evaluations.

3 Results

3.1 Function values and derivative agreement

The implementations in Sections 2.2 and 2.3 are evaluated at $\Delta t = 50$ s using the verification procedure in Section 2.4. Table 2 reports errors against the independent analytical reference. The CTSE and OTI-CR first derivatives have relative errors of approximately 2.54×10^{-14} and 1.64×10^{-14} , respectively. FD agrees through fourth order, with its relative error increasing from 1.39×10^{-11} at first order to 2.10×10^{-5} at fourth order. This reduced accuracy is consistent with the cancellation described in Section 2.4; FD provides corroborating evidence rather than an exact reference.

Table 2: Errors at $\Delta t = 50$ s relative to the independent analytical reference. Order zero denotes the function value. Absolute errors have units K/s^j for derivative order j ; relative infinity errors are dimensionless.

Method	Order	Max. absolute error	Relative infinity error
Original	0	1.78e-11	3.03e-14
CTSE	0	1.38e-11	2.34e-14
CTSE	1	9.81e-14	2.54e-14
OTI-CR	0	1.78e-11	3.03e-14
OTI-CR	1	6.35e-14	1.64e-14
OTI-CR	2	6.80e-15	1.16e-13
OTI-CR	3	9.70e-16	3.72e-13
OTI-CR	4	1.38e-16	8.70e-13
FD	0	1.78e-11	3.03e-14
FD	1	5.37e-11	1.39e-11
FD	2	2.45e-10	4.17e-09
FD	3	1.51e-09	5.81e-07
FD	4	3.33e-09	2.10e-05

Figure 3 shows agreement of the nominal temperatures across the original script, CTSE, OTI-CR, and the real response used as the FD baseline. Figure 4 compares the corresponding voxel sensitivities; CTSE appears only in the first-order panel.

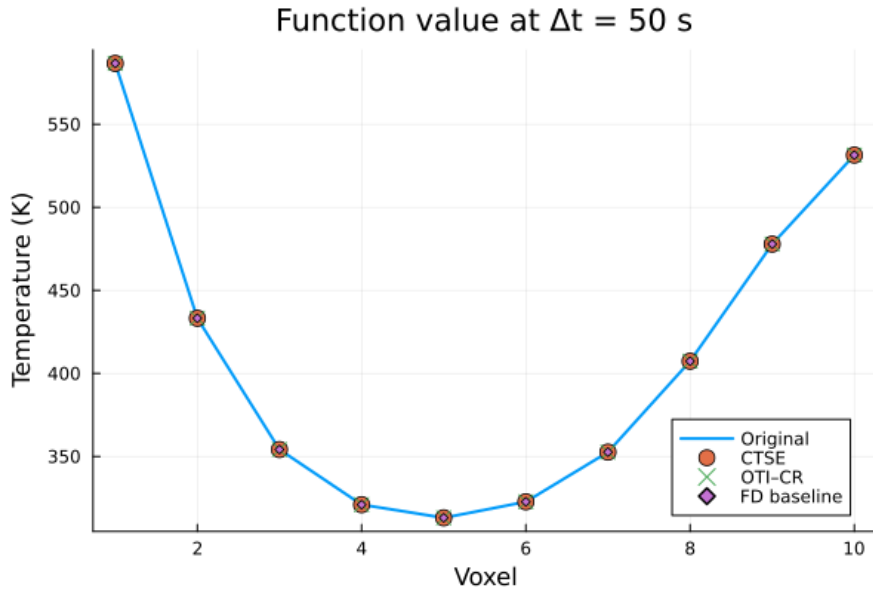


Figure 3: Nominal voxel temperatures at $\Delta t = 50$ s. CTSE and OTI-CR agree with the unchanged original script and the unperturbed real FD baseline.

All 246 test assertions pass. The tests cover scalar derivatives, OTI algebra, zero duration, nonzero forcing, CTSE steps from 10^{-8} to 10^{-30} s, and FD polynomial exactness.

3.2 Derivative profiles and step-size dependence

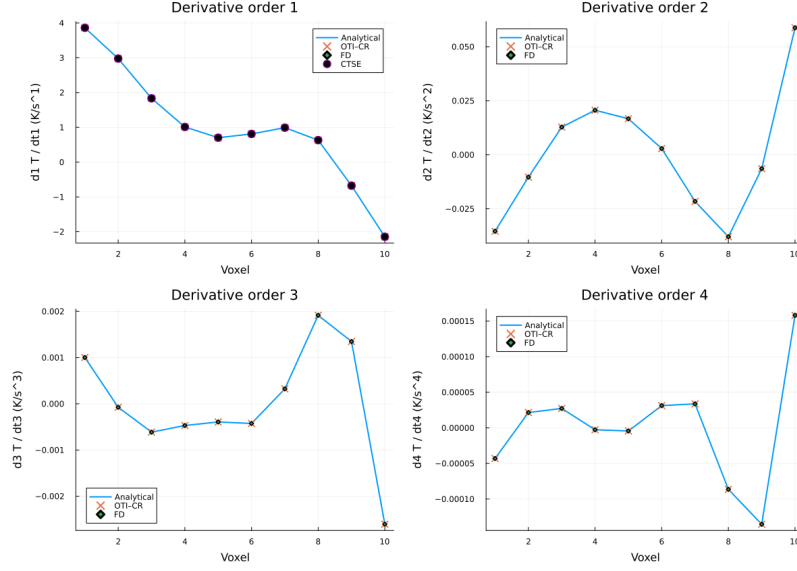


Figure 4: Voxel temperature derivatives at $\Delta t = 50$ s. The upper-left panel compares CTSE, OTI-CR, FD, and the analytical first derivative. CTSE is omitted from orders two through four.

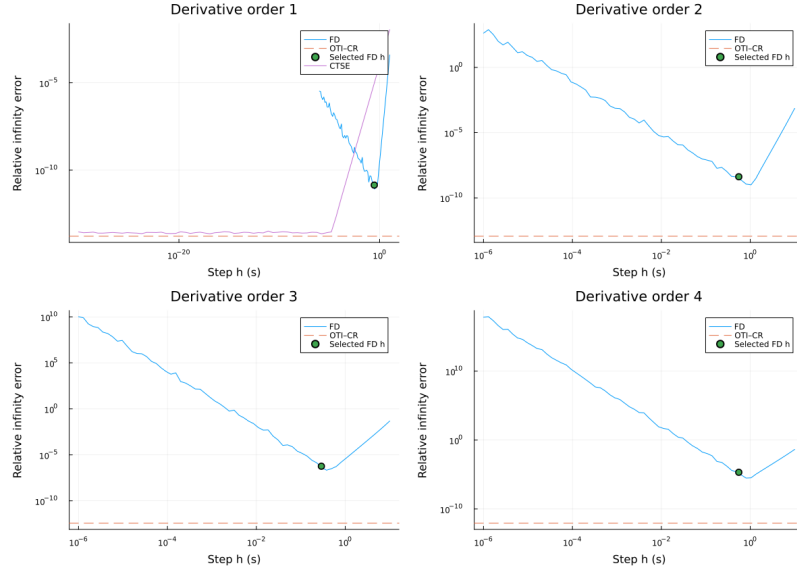


Figure 5: Relative derivative errors versus step size. The upper-left panel compares the CTSE imaginary-step sweep with real finite differences (FD). OTI-CR has no step parameter and is shown as a horizontal reference. Higher-order panels show FD and OTI-CR only.

Figure 5 shows FD accuracy deteriorating for both small and large steps. CTSE maintains first-derivative accuracy for very small imaginary steps in this example. OTI-CR has no step parameter and provides a horizontal error reference. Errors in Figure 5 are floored at machine epsilon only for logarithmic display; the CSV files retain the raw errors.

A Multivariate OTI numbers and their real matrix representation

Sections 1.4 and 1.5 use one parameter and one nilpotent unit. This appendix extends the construction to d independent parameters and explains how one evaluation can carry pure and mixed derivatives. The algebra and matrix representation follow the multivariate OTI construction of [10]; the basis ordering is stated explicitly below.

A.1 Algebra, seeding, and mixed derivatives

Introduce commuting units $\varepsilon_1, \dots, \varepsilon_d$ and retain monomials of *total degree* at most p . For a multi-index $\alpha = (\alpha_1, \dots, \alpha_d)$ of nonnegative integers, define

$$|\alpha| = \alpha_1 + \dots + \alpha_d, \quad \alpha! = \alpha_1! \dots \alpha_d!, \quad \varepsilon^\alpha = \varepsilon_1^{\alpha_1} \dots \varepsilon_d^{\alpha_d}.$$

A multivariate OTI number is

$$a = \sum_{|\alpha| \leq p} a_\alpha \varepsilon^\alpha, \quad a_\alpha \in \mathbb{R}, \quad \varepsilon^\alpha = 0 \text{ if } |\alpha| > p. \quad (\text{A.1})$$

For a second number $b = \sum_{|\alpha| \leq p} b_\alpha \varepsilon^\alpha$, addition combines equal monomials, while multiplication adds their exponents and discards products whose total degree exceeds p :

$$a + b = \sum_{|\alpha| \leq p} (a_\alpha + b_\alpha) \varepsilon^\alpha, \quad ab = \sum_{|\gamma| \leq p} \left(\sum_{\alpha+\beta=\gamma} a_\alpha b_\beta \right) \varepsilon^\gamma. \quad (\text{A.2})$$

For two variables at order two, the retained monomials are $1, \varepsilon_1, \varepsilon_2, \varepsilon_1^2, \varepsilon_1 \varepsilon_2, \varepsilon_2^2$. The mixed term $\varepsilon_1 \varepsilon_2$ is retained, but $\varepsilon_1^2 \varepsilon_2$ and $\varepsilon_1 \varepsilon_2^2$ vanish. Requiring only $\varepsilon_i^{p+1} = 0$ separately would not impose this *total-degree* truncation: mixed monomials above degree p must also vanish.

For an analytic $f(x_1, \dots, x_d)$, seed each parameter independently as $x_i^* = x_{i,0} + \varepsilon_i$. Its multivariate Taylor expansion is

$$f(\mathbf{x}_0 + \varepsilon) = \sum_{|\alpha| \leq p} \frac{\partial^\alpha f(\mathbf{x}_0)}{\alpha!} \varepsilon^\alpha, \quad \partial^\alpha f = \frac{\partial^{|\alpha|} f}{\partial x_1^{\alpha_1} \dots \partial x_d^{\alpha_d}}. \quad (\text{A.3})$$

Extending the coefficient-extraction notation of Equation (11),

$$\boxed{\partial^\alpha f(\mathbf{x}_0) = \alpha! \operatorname{Im}_{\varepsilon^\alpha} (f(\mathbf{x}_0 + \varepsilon))}. \quad (\text{A.4})$$

The scale is the product of factorials $\alpha!$, not the factorial of the total degree $|\alpha|$. For example, the coefficient of ε_1^2 is $f_{x_1 x_1}/2!$, whereas the coefficient of $\varepsilon_1 \varepsilon_2$ is $f_{x_1 x_2}/(1!1!) = f_{x_1 x_2}$. Seeding both inputs with the same unit would instead give a directional derivative and would not keep their partial derivatives separate.

A.2 Building the matrix algebra from its generators

The OTI algebra of Section A.1 has d independent commuting generators $\varepsilon_1, \dots, \varepsilon_d$ and truncation order p . The generators produce all higher-order monomials by multiplication. The full basis consists of the constant monomial 1 and all products of these generators through total degree p . To build a matrix representation, it is therefore sufficient to construct one matrix N_i representing each generator ε_i . The matrix for a higher-order monomial is then the corresponding product:

$$1 \mapsto I_q, \quad \varepsilon_i \mapsto N_i, \quad \varepsilon_1^{\alpha_1} \dots \varepsilon_d^{\alpha_d} \mapsto N_1^{\alpha_1} \dots N_d^{\alpha_d}, \quad (\text{A.5})$$

where $q = \binom{d+p}{p}$ is the number of retained basis monomials. These matrices must reproduce the algebra's defining relations: the N_i commute, and every product of more than p generating factors is zero. The construction below enforces both properties.

Construct each generator matrix by its action on the basis. Choose an ordering of the retained monomials, beginning with 1, and let $[a]$ denote the coordinate vector representing an OTI number a in this order. The number itself remains the algebraic object in Equation (A.1). We define N_i to perform multiplication by ε_i :

$$\boxed{N_i[a] = [\varepsilon_i a].} \quad (\text{A.6})$$

To determine column j , multiply the j th basis monomial by ε_i and express the result in the chosen coordinates. If the resulting total degree exceeds p , that column is zero. Thus every column follows directly from multiplication in the OTI algebra.

Represent a general number by substitution. Once the generator matrices are known, keep each real coefficient and replace its monomial by the matrix product in Equation (A.5):

$$\boxed{C(a) = \sum_{|\alpha| \leq p} a_\alpha N_1^{\alpha_1} \dots N_d^{\alpha_d} = \sum_{|\alpha| \leq p} a_\alpha N^\alpha}, \quad N^0 = I_q. \quad (\text{A.7})$$

Here N^α abbreviates the product of generator matrices. Since the generators commute, these products add multi-indices, $N^\alpha N^\beta = N^{\alpha+\beta}$, with products above total degree p equal to zero, exactly as for the monomials ε^α . Consequently, for any two OTI numbers a and b in the same algebra,

$$C(a+b) = C(a) + C(b), \quad C(ab) = C(a)C(b). \quad (\text{A.8})$$

The first column of $C(a)$ holds the coefficients a_α in the chosen monomial order. The independently seeded input $x_i^* = x_{i,0} + \varepsilon_i$ is represented by $X_i = x_{i,0}I_q + N_i$. As in Section 1.5, analytic elementary functions are evaluated as matrix functions of these representations.

Example: two generators and truncation order two. For $d = 2$, $p = 2$, choose the ordered basis

$$\mathcal{B} = (1, \varepsilon_1, \varepsilon_2, \varepsilon_1^2, \varepsilon_1\varepsilon_2, \varepsilon_2^2).$$

Multiplying each entry of this tuple by a generator gives

$$\begin{aligned}\varepsilon_1\mathcal{B} &= (\varepsilon_1, \varepsilon_1^2, \varepsilon_1\varepsilon_2, 0, 0, 0), \\ \varepsilon_2\mathcal{B} &= (\varepsilon_2, \varepsilon_1\varepsilon_2, \varepsilon_2^2, 0, 0, 0).\end{aligned}\tag{A.9}$$

The last three entries vanish because their total degree would be three. Each output supplies one column of the corresponding matrix, expressed in the coordinates of $\mathcal{B}$. Consequently,

$$N_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad N_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

For example, $\varepsilon_1 \cdot \varepsilon_2 = \varepsilon_1\varepsilon_2$: the third input monomial maps to the fifth basis monomial, placing a one in row five, column three of N_1 . The higher-order monomials are represented by N_1^2 , $N_1N_2 = N_2N_1$, and N_2^2 . Thus there are two generating matrices and six basis matrices,

$$I_6, \quad N_1, \quad N_2, \quad N_1^2, \quad N_1N_2, \quad N_2^2.$$

For a general OTI number

$$a = a_{00} + a_{10}\varepsilon_1 + a_{01}\varepsilon_2 + a_{20}\varepsilon_1^2 + a_{11}\varepsilon_1\varepsilon_2 + a_{02}\varepsilon_2^2,$$

Equation (A.7) becomes

$$\boxed{C(a) = a_{00}I_6 + a_{10}N_1 + a_{01}N_2 + a_{20}N_1^2 + a_{11}N_1N_2 + a_{02}N_2^2.}\tag{A.10}$$

Expanding this linear combination gives

$$C(a) = \begin{bmatrix} a_{00} & 0 & 0 & 0 & 0 & 0 \\ a_{10} & a_{00} & 0 & 0 & 0 & 0 \\ a_{01} & 0 & a_{00} & 0 & 0 & 0 \\ a_{20} & a_{10} & 0 & a_{00} & 0 & 0 \\ a_{11} & a_{01} & a_{10} & 0 & a_{00} & 0 \\ a_{02} & 0 & a_{01} & 0 & 0 & a_{00} \end{bmatrix}.\tag{A.11}$$

The first column agrees with $[a]$, as required. Every other column records multiplication of a by the corresponding basis monomial. These matrices form a six-dimensional subalgebra of the real 6×6 matrices.

A.3 Worked example with pure and mixed derivatives

Extend the scalar example of Section 1.4.1 to $f(x, y) = \exp(x^2 + y)$ at $(x_0, y_0) = (1, 0)$, retaining total degree two. The independently seeded inputs and the argument of the exponential are

$$x^* = 1 + \varepsilon_1, \quad y^* = \varepsilon_2, \quad (x^*)^2 + y^* = 1 + 2\varepsilon_1 + \varepsilon_1^2 + \varepsilon_2.$$

The Taylor expansion of the exponential about its real part 1 gives

$$f(x^*, y^*) = e \left[1 + (2\varepsilon_1 + \varepsilon_1^2 + \varepsilon_2) + \frac{(2\varepsilon_1 + \varepsilon_1^2 + \varepsilon_2)^2}{2} \right],$$

$$(2\varepsilon_1 + \varepsilon_1^2 + \varepsilon_2)^2 = 4\varepsilon_1^2 + 4\varepsilon_1\varepsilon_2 + \varepsilon_2^2.$$

All omitted products have total degree at least three. Therefore

$$f(x^*, y^*) = e \left(1 + 2\varepsilon_1 + \varepsilon_2 + 3\varepsilon_1^2 + 2\varepsilon_1\varepsilon_2 + \frac{1}{2}\varepsilon_2^2 \right). \quad (\text{A.12})$$

For the real matrix evaluation, use $X = I_6 + N_1$ and $Y = N_2$. The identical sequence of operations is a matrix product followed by a matrix exponential:

$$V = X^2 + Y = I_6 + 2N_1 + N_1^2 + N_2,$$

$$W = \exp(V) = e \left(I_6 + 2N_1 + N_2 + 3N_1^2 + 2N_1N_2 + \frac{1}{2}N_2^2 \right). \quad (\text{A.13})$$

The second equality follows by expanding about I_6 ; products of three nilpotent factors vanish. The first column is

$$W\mathbf{e}_0 = \begin{bmatrix} e & 2e & e & 3e & 2e & e/2 \end{bmatrix}^T.$$

Reading the entries in the declared monomial order and applying Equation (A.4) yields

$$f(1, 0) = e, \quad \nabla f(1, 0) = \begin{bmatrix} 2e \\ e \end{bmatrix}, \quad \nabla^2 f(1, 0) = \begin{bmatrix} 6e & 2e \\ 2e & e \end{bmatrix}. \quad (\text{A.14})$$

In particular, the mixed derivative is the fifth entry without an extra factor of two, while the fourth and sixth entries are multiplied by 2!. Direct differentiation of $\exp(x^2 + y)$ confirms all these values.

A.4 General matrices with multivariate OTI entries

The Kronecker construction of Equation (25) extends by replacing powers of one shift matrix with products of the N_i :

$$M = \sum_{|\alpha| \leq p} M_\alpha \epsilon^\alpha \quad \mapsto \quad \boxed{\mathcal{C}(M) = \sum_{|\alpha| \leq p} N^\alpha \otimes M_\alpha}. \quad (\text{A.15})$$

If $M_\alpha \in \mathbb{R}^{m \times m}$, the representation is a single real $qm \times qm$ matrix. The coefficient matrices may be arbitrary and need not commute. Their order is preserved by $(N^\alpha \otimes M_\alpha)(N^\beta \otimes L_\beta) = N^{\alpha+\beta} \otimes (M_\alpha L_\beta)$, with products above total degree p equal to zero. Consequently, the representation preserves matrix multiplication and the matrix exponential.

For example, let $M(x, y) = xA + yB$, with fixed real $m \times m$ matrices A, B , and let $D = x_0 A + y_0 B$. Seeding both parameters at total degree two gives

$$\mathcal{M} = I_6 \otimes D + N_1 \otimes A + N_2 \otimes B = \begin{bmatrix} D & 0 & 0 & 0 & 0 & 0 \\ A & D & 0 & 0 & 0 & 0 \\ B & 0 & D & 0 & 0 & 0 \\ 0 & A & 0 & D & 0 & 0 \\ 0 & B & A & 0 & D & 0 \\ 0 & 0 & B & 0 & 0 & D \end{bmatrix}. \quad (\text{A.16})$$

Every displayed entry is an $m \times m$ block. For $F(x, y) = \exp(xA + yB)$, the first block column of $\exp(\mathcal{M})$ is

$$\begin{bmatrix} F \\ F_x \\ F_y \\ F_{xx}/2! \\ F_{xy} \\ F_{yy}/2! \end{bmatrix} \bigg|_{(x_0, y_0)}. \quad (\text{A.17})$$

No assumption that $AB = BA$ is required; in general the derivatives cannot be simplified to products such as $A \exp(D)$ or $AB \exp(D)$.

Verification and scope. The companion script below verifies the two-variable multiplication matrices, the scalar example in Equation (A.14), and the block construction with a noncommuting pair A, B :

```
julia --project=. docs/verify_multivariate_example.jl
```

The thermal sensitivity entry points remain single-parameter implementations. This appendix supplies the multivariate construction and a separate numerical check. For d parameters at total order p , $q = \binom{d+p}{p}$ grows with both d and p , so the dense real representation becomes substantially larger. It should not be replaced by an unmodified tensor product of univariate representations: that would retain terms up to degree p in each variable, rather than the total-degree truncation specified in Section A.1.

References

- [1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind (2018). [Automatic Differentiation in Machine Learning: a Survey](#). *Journal of Machine Learning Research*, 18(153), 1–43.
- [2] C. H. Bischof, H. M. Bücker, P. Hovland, U. Naumann, and J. Utke (eds., 2008). [Advances in Automatic Differentiation](#). Springer.
- [3] A. Griewank and A. Walther (2008). [Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation](#). Second edition, SIAM.
- [4] E. Phipps and R. Pawlowski (2012). Efficient Expression Templates for Operator Overloading-Based Automatic Differentiation. In *Recent Advances in Algorithmic Differentiation*, Springer, 309–319.
- [5] MDO Lab. [A Guide to the Complex-Step Derivative Approximation](#).
- [6] J. R. R. A. Martins, P. Sturdza, and J. J. Alonso (2003). [The Complex-Step Derivative Approximation](#).
- [7] G. Lantoiné, R. P. Russell, and T. Dargent (2012). [Using Multicomplex Variables for Automatic Computation of High-Order Derivatives](#). *ACM Transactions on Mathematical Software*, 38(3), article 16.
- [8] J. Fike and J. Alonso (2011). The Development of Hyper-Dual Numbers for Exact Second-Derivative Calculations. *49th AIAA Aerospace Sciences Meeting*, Orlando, Florida.
- [9] N. J. Higham (2008). *Functions of Matrices: Theory and Computation*. SIAM.
- [10] M. Aristizabal Cano (2020). [Order truncated imaginary algebra for computation of multivariable high-order derivatives in finite element analysis](#).